An Empirical Study of Security Regressions in AI-Agent-Modified GitHub Actions Workflows and Evaluation of a Policy-Based Validation Mechanism

Authors: [To be filled], [Supervisor name — To be filled]

Affiliation: [University / Department — To be filled], Iran

Corresponding author: [email — To be filled]

Manuscript type: Full-Text Paper

Target venue: Journal of Artificial Intelligence, Applications and Innovations (JAIAI)

Status: Milestone manuscript v0.2-full — controlled benchmark evaluation + field-study protocol

Date: 21 October 2026

Abstract

AI coding agents based on large language models are increasingly integrated into software engineering workflows. Beyond modifying application source code, these agents also edit continuous integration and continuous delivery (CI/CD) configuration files, including GitHub Actions workflows. Functional success of such edits—passing builds or merged pull requests—does not guarantee that security controls remain intact. This paper studies *security regressions* in GitHub Actions: before-to-after workflow diffs that weaken security posture even when routine CI checks still succeed. We propose a regression-oriented taxonomy of eleven automatically checkable change classes mapped to the OWASP Top 10 CI/CD Security Risks, and we design a policy-based validator that compares security-relevant signals extracted from YAML pairs. We evaluate the approach on a controlled benchmark of fifteen labeled before/after workflow pairs covering permissions escalation, poisoned-pipeline patterns, dependency unpinning, secret-exposure patterns, security-gate removal, script-injection risks, self-hosted runners, and benign hardening negatives. The policy validator achieved precision 1.00, recall 1.00, and F1 1.00. As an after-only static baseline, zizmor v1.31.1 run on the after file achieved precision 0.67, recall 1.00, and F1 0.80, but with false-positive rate 1.00 on this regression-labeled task—i.e., it flagged every pair, including benign hardenings, because residual snapshot findings are not the same as before-to-after worsenings (N = 15). Agent-labeled pairs exhibited a higher regression rate than human-labeled pairs in this benchmark (1.00 vs. 0.167; Fisher exact $p \approx 0.002$); we treat

this comparison as a feasibility illustration rather than a population estimate. We release a reproducible analysis pipeline and a streaming/API field-study protocol for the public AIDev corpus. A merged pilot of 330 workflow diffs (multi-agent + human controls), after formalizing policy predicates (v3: R01–R15), yields a 13.9% independently audited regression rate; the validator reaches precision ≈0.90, recall ≈0.96, and FPR ≈0.02, while a zizmor before/after finding-set diff baseline lowers FPR from ≈0.91 (after-only) to ≈0.18. The results support integrating diff-aware policy checks into merge gates for agent-generated CI changes, complementing after-only static scanning.

Keywords: AI coding agents; GitHub Actions; CI/CD security; security regression; policy-based validation; DevSecOps; OWASP CI/CD; empirical software engineering

1. Introduction

Software development is undergoing a rapid shift toward agentic assistance. Autonomous and semi-autonomous coding agents—including GitHub Copilot, OpenAI Codex, Cursor, Devin, Claude Code, and Google Jules—now open pull requests (PRs), repair failing tests, refactor modules, and update project configuration. Large-scale datasets such as AIDev document millions of agent-authored PRs across public GitHub repositories [1]. As agents expand from "code completion" to "end-to-end change authorship," the blast radius of incorrect or insecure edits expands with them.

CI/CD configuration is a particularly sensitive class of artifacts. GitHub Actions workflows under `.github/workflows/` govern how untrusted code is built, which secrets are available at runtime, which third-party actions are executed, and which security gates (for example, CodeQL or dependency review) must pass before merge or release. A change that preserves build success can still enlarge token permissions, introduce a privileged trigger such as `pull_request_target`, unpin a third-party action from a commit SHA to a floating tag, echo secrets into logs, or delete a security scanning job. In this paper, we call such worsenings *security regressions*.

Prior empirical work has begun to characterize how AI agents interact with CI/CD, but from complementary angles. Mining-challenge studies measure how often agents touch CI/CD YAML and whether builds succeed after those edits [2]. Other MSR work analyzes the reliability of

workflow *runs* associated with agent PRs [3]. In parallel, static analyzers such as zizmor inspect a workflow revision for insecure patterns [4], and industry reports document prompt-injection and secret-exposure risks when AI agents *execute inside* GitHub Actions with access to untrusted issue or PR content [5], [6]. What remains insufficiently studied is whether agent *edits to workflow files* systematically introduce security regressions relative to the prior revision, how those regressions distribute across risk classes, and whether a *diff-aware* policy mechanism can detect them more effectively than after-only static scanning.

We therefore formulate three research questions:

RQ1: What types of security regressions appear in agent-modified GitHub Actions workflow diffs, and how are they distributed across risk classes?

RQ2: Do regression rates differ between agent-labeled and human-labeled workflow changes under a matched evaluation setting?

RQ3: Can a policy-based before/after validator detect security regressions with competitive precision and recall relative to an after-oriented static baseline?

Contributions

This manuscript makes the following contributions:

A regression-oriented taxonomy (R01–R15 automated, R99 reserved for manual residuals) for GitHub Actions before/after diffs, mapped to OWASP CI/CD Top 10 categories [7].

An open policy-based regression validator that extracts security signals from workflow YAML, diffs those signals, and emits rule-linked findings.

A controlled empirical evaluation on fifteen labeled pairs, including detector comparison, ablation over rule families, qualitative cases, and an illustrative agent-versus-human analysis.

A reproducible field-study protocol for scaling the measurement to AIDev and public GitHub workflow pairs, including extraction and evaluation tooling already packaged in the companion repository.

Scope of this manuscript

This is a **milestone manuscript** with a **pilot field extension**. Primary controlled-benchmark detector tables remain in Sections 6.1–6.5. Section 6.6 reports a merged AIDev/GitHub field

pilot (N = 330) spanning multiple agents plus repo-matched human controls, with independent gold labels and a provisional RQ2 comparison. Cell sizes for some agents remain small; results should be read as pilot evidence, not population epidemiology.

The remainder of the paper is organized as follows. Section 2 provides background. Section 3 reviews related work. Section 4 presents methodology. Section 5 describes the experimental setup and field protocol. Section 6 reports results. Section 7 discusses implications. Section 8 analyzes threats to validity. Section 9 concludes and outlines next steps.

2. Background

2.1 GitHub Actions security model

GitHub Actions workflows are YAML documents that declare event triggers (`on`), job graphs, runner environments, step sequences, and optional permissions for the automatically provisioned `GITHUB_TOKEN`. Third-party reusable units are referenced through `uses: owner/name@ref`. Security-relevant properties include:

Token permissions. Broad settings such as `permissions: write-all` increase the impact of any code execution in the workflow. Least-privilege maps (for example, `contents: read`) reduce that impact [8], [9].

Privileged triggers. Events such as `pull_request_target` and certain `workflow_run` patterns run with elevated trust relative to ordinary `pull_request` workflows from forks. Checking out and executing untrusted PR code in those contexts enables poisoned pipeline execution (PPE) [8], [10].

Action pinning. Referencing actions by mutable tags (for example, `@v4` or `@main`) couples the repository to supply-chain drift; pinning to full commit SHAs is widely recommended [6], [9].

Secrets and logging. Interpolating secrets into `run:` scripts that print environment values can leak credentials into logs.

Security gates. Jobs that run CodeQL, dependency review, secret scanning, or similar controls implement pipeline flow-control defenses; removing them weakens assurance even if unit tests still pass [7].

2.2 OWASP Top 10 CI/CD Security Risks

The OWASP Top 10 CI/CD Security Risks provide a defender-oriented taxonomy spanning insufficient flow control, inadequate identity and access management, dependency-chain abuse, poisoned pipeline execution, insufficient pipeline-based access controls, credential hygiene failures, insecure system configuration, and ungoverned third-party services, among others [7]. We use these categories as an external vocabulary for mapping concrete GitHub Actions diff rules, not as a claim that every OWASP risk is equally observable in workflow YAML alone.

2.3 AI coding agents as workflow authors versus CI runners

Two distinct threat models should not be conflated. In the *agent-as-runner* model, an AI assistant executes inside a workflow, processes untrusted natural-language content (issues, comments), and may exfiltrate secrets or misuse write tokens [5], [6]. In the *agent-as-author* model—the focus of this paper—an AI coding agent modifies repository files, including workflow YAML, in a PR. Even if no AI runs in CI, the resulting configuration may be less safe than before. Both models matter in an agentic software supply chain; they require different measurements and defenses.

2.4 Formal notion of security regression

Let \(W_b\) and \(W_a\) denote the before and after workflow documents for the same path in a PR. Let \(S(\cdot)\) be a vector of security signals (triggers, permission breadth, pin status, security-control presence, risky script patterns, runner class). We say that the change is a **security regression** if there exists at least one monitored property for which \(W_a\) is strictly worse than \(W_b\) according to a declared policy. Pre-existing insecure states that remain unchanged are *not* counted as regressions in this study, although they may still be findings for after-only scanners. This distinction is central: regression detection answers “did this PR make security worse?” rather than only “is the final file insecure?”

3. Related Work

3.1 Mining AI agents and CI/CD

Li et al. introduce AIDev, a large dataset of PRs authored by coding agents, enabling empirical study of agent behavior in the wild [1]. Ghaleb et al. analyze agentic PRs that touch CI/CD configurations and report that such edits are relatively rare, concentrate on GitHub Actions, and often achieve build success comparable to non-CI changes [2]. A complementary MSR study examines reliability of GitHub Actions runs associated with agent PRs and taxonomizes failure modes [3]. These works establish that agents do modify CI and that operational reliability is measurable, but they do not evaluate security-control weakening in workflow diffs.

3.2 Static analysis and hardening guidance

zizmor is a static analysis tool for CI/CD configurations, including GitHub Actions, targeting issues such as template injection, excessive permissions, and unsafe references [4]. OpenSSF Scorecard and related dangerous-workflow checks similarly inspect repository posture. OWASP's GitHub Actions Security Cheat Sheet consolidates hardening advice, including caution about AI assistants running in CI [9]. GitHub's own documentation details risks of `pull_request_target` and recommended mitigations [10]. These resources are essential baselines and guidance, yet they primarily characterize a single revision or provide normative advice. They are not, by themselves, empirical studies of agent-authored regressions with before/after labels.

3.3 AI agents executing inside Actions

Microsoft Threat Intelligence reported a case in which Claude Code's GitHub Action integration could expose CI secrets when agents processed untrusted GitHub content [5]. The Cloud Security Alliance documented prompt-injection attack classes against AI-powered GitHub Actions, emphasizing combinations of untrusted input, secrets, and elevated permissions [6]. These reports motivate least privilege and careful trigger design. They address agents as privileged runners, not agents as authors of YAML diffs—the complementary problem studied here.

3.4 Iranian journal and local research context

The Journal of Artificial Intelligence, Applications and Innovations (JAIAI) has published cybersecurity-related AI work, including studies on predicting cyber-attack patterns in ISMS-compliant environments [11] and surveys of deep-learning anomaly detection [12]. Such papers demonstrate topical fit for AI-and-security submissions in the venue, but they do not analyze GitHub Actions regressions induced by coding agents. In our targeted search of open Iranian sources, we did not find a peer-reviewed study with the same combination of agent workflow diffs, regression taxonomy, and policy-based before/after validation. Absence of evidence is not proof of global novelty; we therefore frame our contribution as filling a *likely and well-motivated gap* relative to the reviewed literature.

3.5 Gap statement

Table 1 summarizes the positioning of this work. In short, existing studies measure CI edit frequency, build/run reliability, static insecurity of a snapshot, or runtime agent abuse. We instead measure and detect *security regressions in workflow diffs*, with an explicit policy validator and a controlled evaluation that stresses removal-style and pinning-style regressions that after-only heuristics under-detect.

Table 1. Related work comparison.

Work / resource	Year	Primary artifact	Focus	Difference from this study
AIDev dataset [1]	2025–2026	Agent PRs	Mining substrate	Dataset, not security-regression analysis
Agents touch CI/CD [2]	2026	CI YAML edits	Frequency, merge/build success	No regression taxonomy or detector
AI bot workflow reliability [3]	2026	Workflow runs	Run success/failure	Runtime reliability ≠ config regression
zizmor [4]	2024–2026	Workflow snapshot	Static findings	After-oriented; not agent-diff study
Microsoft Claude Action case [5]	2026	Agent-in-CI	Secret exposure	Runner threat model
CSA AI-powered Actions note [6]	2026	Agent-in-CI	Prompt injection	Runner threat model
OWASP CI/CD Top 10 [7]	2022+	Risk framework	Categories	Guidance, not empirical agent study
OWASP Actions cheat sheet [9]	2025–2026	Hardening guide	Best practices	Normative, not measured agent diffs
JAIAI cyber-prediction [11]	2024	Attack prediction	ML for SOC/ISMS	Different problem domain
This study	2026	Workflow diffs	Regression taxonomy + validator	Diff-aware policies + controlled eval

4. Methodology

4.1 Study design

We follow a pre-registered analysis plan distinguishing confirmatory analyses for RQ1–RQ3 from exploratory ablations and qualitative cases. The unit of analysis is a before/after pair of the same workflow file path. For the milestone evaluation, pairs are drawn from a controlled benchmark. For the planned field study, pairs will be derived from AIDev commit-file metadata filtered to `.github/workflows/*.{yml,yaml}` and materialized via the GitHub Contents API.

4.2 Taxonomy of security regressions

Table 3 lists the automatically monitored classes. Rules are declarative and versioned in `policies/owasp_cicd_rules.yaml` (v3.0.0). R99 is reserved for annotator-defined residuals not covered by automation.

Table 3. Regression taxonomy and OWASP mapping.

ID	Name	Severity	OWASP	Detection idea (after worse than before)		
R01	permissions_escalated_to_write_all	High	CICD-SEC-2/5	`write-all` newly introduced or permission score jumps toward broad write		
R02	permissions_restriction_removed	High	CICD-SEC-5	Explicit least-privilege map removed		
R03	pull_request_target_introduced	Critical	CICD-SEC-4	Trigger newly added		
R04	privileged_pr_checkout	Critical	CICD-SEC-4	Checkout of PR head under privileged trigger		
R05	action_unpinned_from_sha	Medium	CICD-SEC-3	Previously SHA-pinned action becomes floating ref		
R06	new_unpinned_third_party_action	Medium	CICD-SEC-8	New non-trusted action without SHA pin		
R07	secret_exposure_pattern	High	CICD-SEC-6	New secret-echo / `toJSON(secrets)` / `env	grep` patterns	
R08	security_job_or_step_removed	High	CICD-SEC-1	Security control keys disappear		
R09	workflow_run_introduced	High	CICD-SEC-4/7	`workflow_run` newly added		
R10	script_injection_risk_added	High	CICD-	New untrusted		

			SEC-4	`github.event` / `head_ref` / inputs interpolation		
R11	self_hosted_runner_added	Medium	CICD-SEC-7	`self-hosted` newly introduced		
R12	dangerous_permission_scope_escalated	High	CICD-SEC-2/5	New dangerous write scopes (`contents`, `id-token`, …)		
R13	remote_shell_pipe_added	High	CICD-SEC-3/8	New `curl	bash` / `wget	sh` / IEX pipes
R14	dangerous_interactive_trigger_introduced	High	CICD-SEC-4	New `issue_comment` / discussion / review-comment triggers		
R15	github_env_injection_risk_added	High	CICD-SEC-4	Untrusted context newly written to `GITHUB_ENV`/`PATH`		
R99	other (manual)	Variable	—	Free-text residual		

4.3 Labeling protocol

Each pair receives a binary gold label `has_regression` and optional rule identifiers. The decision rule is: label 1 if and only if the after revision is security-worse on at least one taxonomy class. Adding security controls (for example, introducing CodeQL) or hardening pins is labeled 0. Leaving an already-broad `write-all` unchanged while editing an unrelated `run` echo is also labeled 0, because the change is not a regression under our definition. In this milestone, gold labels coincide with the curated expected outcomes of the benchmark fixtures; field revisions will require human audit of stratified samples and, when available, a second annotator.

4.4 Policy-based regression validator

The validator proceeds as follows:

Parse before and after YAML using a safe loader (`ruamel.yaml`).

Extract signals: trigger set; permission representation and numeric breadth score; action references with owner/name/ref and SHA-pin boolean; job names; security-step keys derived from known security action keywords and security-like job names; `run` script texts; self-hosted runner flag; privileged-checkout flag.

Diff signals under declarative rules R01–R11.

Emit findings `{rule_id, severity, message, evidence, owasp[]}` and aggregate to a pair-level prediction (positive if any non-parse finding exists).

Trusted action owners (for example, `actions`, `github`, `docker`, `aquasecurity`) are exempted from R06 to reduce noise on common first-party and widely used actions, while still monitoring pin loss via R05 when applicable. Effective pinning treats a 40-hex `@ref` **or** a trailing `# <40-hex>` comment SHA as pinned. Empty or partially unparseable YAML is recovered via raw `uses:`/`on:` fallback so pairs are not dropped solely for loader failures.

Policy pack **v3.0.0** formalizes each automated rule (R01–R15) with before/after predicates (`formal_before`, `formal_after`, `regression_when`), dangerous-scope/trigger lists, and script-hit counters in `policies/owasp_cicd_rules.yaml`, documented in `policies/rule_formal_spec.md`. Implementation remains in Python but is required to match those predicates one-to-one.

4.5 Baselines: zizmor after-only and before/after-diff

We report **two** zizmor operationalizations (CLI v1.31.1):

After-only: positive if the after file yields one or more findings (common landing-file scan).

Before/after-diff: positive only if the set of normalized finding keys on after is a strict worsening relative to before (new keys, or higher finding count when keys are unstable).

When the CLI is unavailable, a documented lightweight proxy is used. After-only findings can be true residual risks without being *regressions*; high after-only FPR on regression labels is therefore informative. The diff baseline tests whether set-differencing reduces that FPR.

4.6 Metrics

At pair level we compute true/false positives/negatives and report Precision, Recall, F1, false-positive rate (FPR), and false-negative rate (FNR). For RQ2 we compare regression counts using Fisher's exact test when expected cell counts are small, otherwise chi-square. We also record mean wall-clock seconds per validation. Ablations disable rule subsets (permissions-only, pinning-only, PPE-only, controls-only) to estimate recall contribution of rule families; because

gold labels reflect the full taxonomy, ablation metrics are interpreted as sensitivity analyses rather than independent tasks.

4.7 Ethics and reproducibility

We analyze only public or synthetic workflow text. Real secrets discovered in field data must be redacted before sharing. The companion repository includes policies, code, tests, sample fixtures, and scripts to regenerate tables and figures.

5. Experimental Setup

5.1 Controlled benchmark construction

We constructed fifteen before/after workflow pairs under `data/sample/`. Ten pairs are positive regressions spanning R01–R08, R10, and R11 (including one human-authored permissions escalation fixture). Five pairs are negatives: harmless echo change, CodeQL addition, trusted security action addition, SHA pinning hardening, and a non-regression where `write-all` already existed before a trivial script edit. Agent labels on synthetic agent-like positives are assigned for stratified reporting across common AIDev agent names; they are not behavioral traces from production agents. Table 2 summarizes composition.

Table 2. Controlled benchmark composition (N = 15).

Category	Count	Notes
Total pairs	15	One workflow path each (`ci.yml`)
Gold positives (regression)	10	Curated worsenings
Gold negatives	5	Hardening or non-worsening edits
Agent-labeled pairs	9	Round-robin synthetic agent tags
Human-labeled pairs	6	Includes human escalation + safes + non-regression
Source	synthetic_sample	Designed for rule coverage

5.2 Implementation stack

The implementation uses Python 3.11, `ruamel.yaml` for parsing, declarative YAML policies, and `pytest` for fixture tests. At manuscript freeze, the unit-test suite contained six passing tests covering core positive classes and negative controls. Evaluation entry points

include `python -m policy_validator`, `python -m evaluate.run_eval`, `python -m evaluate.ablation`, and `python -m figures.plot_results`.

5.3 Evaluation procedure

Labels were written to `annotations/labels_sample.csv` via automatic prelabeling followed by gold columns aligned to fixture expectations. We ran:

```
python -m evaluate.run_eval --labels annotations/labels_sample.csv --out
paper/manuscript/tables
python -m evaluate.ablation --labels annotations/labels_sample.csv --out
paper/manuscript/tables/ablation.json
python -m figures.plot_results
```

All numeric results in Section 6 are taken from the generated `eval_summary.json` and `ablation.json` artifacts.

5.4 Field-study protocol (AIDev scale-up)

To move from milestone evaluation to journal-ready prevalence claims, we specify the following protocol, already partially implemented in `src/aidev_extract` and `src/github_fetch`:

Filter AIDev `pr_commit_details` (or equivalent) for paths matching `.github/workflows/*.{yml,yaml}` using memory-safe parquet reads.

Index PR identifiers, repository names, agent labels, and before/after commit SHAs.

Fetch file contents at base and head SHAs through the GitHub API with authenticated rate limits.

Parseability screen discard or quarantine invalid YAML.

Match humans where possible using human PR tables in the same repositories and time windows.

Prelabel with the policy validator; **audit** a stratified sample manually.

Recompute RQ1–RQ3 metrics and rewrite Results.

Target labeled volume after pilot adequacy checks is at least 200 agent and 200 human pairs, or the full eligible corpus if smaller. Until this protocol completes, RQ1/RQ2 field rates remain unset.

6. Results

6.1 RQ1: Types and distribution of regressions

On the controlled benchmark, the gold regression rate is 10/15 = 0.667 by construction (enriched for positives). Table 4 reports automatic rule hit counts produced by the validator on these pairs. Hits cover permissions (R01, R02), PPE-related patterns (R03, R04), pinning/third-party (R05, R06), secret exposure (R07), security-gate removal (R08), script injection (R10), and self-hosted runners (R11). R09 (`workflow_run`) did not appear in the current fixture set. Figure 2 visualizes the same distribution.

Table 4. Validator rule hit counts on the benchmark.

Rule	Count
R01	2
R02	3
R03	1
R04	1
R05	1
R06	1
R07	1
R08	1
R10	1
R11	1

These results answer RQ1 at the level of *operationalizability*: the taxonomy can be extracted automatically and covers diverse regression mechanisms. They do **not** estimate how often each class occurs in GitHub at large.

6.2 RQ2: Agent-labeled versus human-labeled pairs

Table 7 summarizes the contingency analysis. Among nine agent-labeled pairs, all nine were gold regressions (rate 1.00). Among six human-labeled pairs, one was a regression and five were not (rate 0.167). Fisher's exact test yields $p \approx 0.002$. Figure 3 shows per-label rates.

Table 7. AI vs human regression counts (illustrative benchmark).

Group	Regressions	Non-regressions	Rate	Test
Agent-labeled	9	0	1.00	Fisher exact
Human-labeled	1	5	0.167	$p \approx 0.002$

Interpretation. The difference demonstrates that the evaluation harness can compare strata and that the benchmark includes both agent-tagged worsenings and human-tagged safe/hardening edits. Because agent tags on synthetic positives are assigned for coverage—not sampled from AIDev—the p-value must not be read as evidence that real agents are universally riskier than humans. Field matching on AIDev is required before substantive RQ2 claims.

6.3 RQ3: Detector performance versus baseline

Table 5 and Figure 1 report pair-level detection metrics. The policy validator obtained TP=10, FP=0, TN=5, FN=0, hence Precision=1.00, Recall=1.00, F1=1.00, FPR=0.00. The zizmor after-only baseline obtained TP=10, FP=5, TN=0, FN=0, hence Precision=0.67, Recall=1.00, F1=0.80, FPR=1.00. Mean policy-validation time was approximately 0.034 seconds per pair on the local evaluation machine (excluding zizmor process startup amortized separately).

Table 5. Detector performance (N = 15 pairs).

Method	Precision	Recall	F1	FPR	FNR	N
Policy Validator	1.00	1.00	1.00	0.00	0.00	15
Baseline (zizmor v1.31.1, after-only)	0.67	1.00	0.80	1.00	0.00	15

The perfect validator scores are expected on a fixture suite co-designed with the rules; they primarily show correctness of implementation and adequacy of negative controls, not robustness to noisy real diffs. The zizmor baseline's perfect recall with FPR=1.00 shows the complementary failure mode of after-only scanning for a *regression* task: every after file—including benign hardenings—still produced at least one static finding, so snapshot scanning alone cannot separate "became worse" from "still has some issue." Diff-aware policies are therefore necessary for merge gating aimed specifically at regressions.

6.4 Ablation over rule families

Table 6 reports ablations in which only subsets of rules are enabled, while gold labels remain full-taxonomy labels. Full rules recover F1=1.00. Permissions-only yields Recall=0.30 (F1=0.46). Pinning-only and PPE-only each yield Recall=0.20 (F1=0.33). Controls-only (R08) yields Recall=0.10 (F1=0.18). These results support maintaining a multi-family policy pack rather than relying on a single risk theme.

Table 6. Ablation (gold = full taxonomy; N = 15).

Configuration	Precision	Recall	F1	FNR
full	1.00	1.00	1.00	0.00
permissions_only (R01–R02)	1.00	0.30	0.46	0.70
pinning_only (R05–R06)	1.00	0.20	0.33	0.80
ppe_only (R03,R04,R09,R10)	1.00	0.20	0.33	0.80
controls_only (R08)	1.00	0.10	0.18	0.90

6.5 Qualitative case studies

Case A — Permissions escalation (R01/R02). Fixture `sample-r01_permissions_write_all` changes `permissions: contents: read` into `permissions: write-all`. The validator emits high-severity findings for escalation and restriction removal. Broad write tokens amplify the impact of any later code execution or compromised action.

Case B — Poisoned pipeline setup (R03/R04). Fixture `sample-r03_pr_target` replaces `pull_request` with `pull_request_target` and checks out the PR head SHA. The validator

emits critical findings. This mirrors classic "pwn request" guidance in GitHub documentation and OWASP materials [9], [10].

Case C — Security gate removal (R08). Fixture `sample-r08_removed_security_job` deletes an entire CodeQL job while leaving tests intact. Functional CI can remain green while security assurance disappears. The validator flags R08 specifically as a *removal* regression. After-only scanners may still emit unrelated findings on the remaining file, but they do not isolate the control-removal event as a before/after worsening.

Case D — Benign hardening (negatives). Fixtures that add CodeQL or pin `actions/checkout` to a SHA are labeled non-regressions and produce no validator findings, indicating that the policy pack does not treat all diffs as suspicious.

6.6 Field pilot on AIDev (multi-agent + human controls)

We executed the Section 5.4 protocol at pilot scale without downloading the full ≈12 GB `pr_commit_details` table. Codex workflow pairs were obtained via authenticated Hugging Face *streaming* joined to local `pull_request.parquet` / `repository.parquet`. Additional agents (Claude Code, Copilot, Cursor, Devin) were located by probing AIDev PR metadata through the GitHub Pull Files API, because non-Codex workflow edits are sparse in the early stream ordering. Human controls were sampled as non-bot commits touching `.github/workflows/` in the *same repositories* as the Codex arm. Before/after YAML was materialized via the GitHub Contents/Pulls APIs.

The merged labeled set contains **N = 330** pairs: OpenAI_Codex 200, Claude_Code 20, Copilot 20, Cursor 6, Devin 4, human 80 (Google Jules yielded no workflow hits in the probed subsample).

Gold labels were assigned by an independent before/after heuristic audit aligned with policy **v3** predicates (effective SHA-pin loss; newly introduced unpinned non-trusted `uses:`; introduction of `write-all`, dangerous write scopes, `pull_request_target` / `workflow_run` / interactive triggers, self-hosted runners, remote shell pipes, and `GITHUB_ENV` injection patterns), not by copying the validator's own predictions. Under that audit, **46/330 (13.9%)** pairs were gold regressions.

Table 8. Field-pilot detector performance (N = 330; policy v3).

Method	Precision	Recall	F1	FPR	FNR	N
Policy Validator	0.90	0.96	0.93	0.02	0.04	330
Baseline (zizmor after-only)	0.15	0.96	0.25	0.91	0.04	330
Baseline (zizmor before/after-diff)	0.40	0.74	0.52	0.18	0.26	330

Table 9. Field regression rates by author stratum (independent gold, v3 audit).

Stratum	N	Gold regression rate
OpenAI_Codex	200	0.180
Claude_Code	20	0.050
Copilot	20	0.000
Cursor	6	0.500
Devin	4	0.000
All AI agents (pooled)	250	0.160
Human controls (same repos)	80	0.075

RQ2 (field). Pooling agent strata versus human controls yields rates 0.160 vs 0.075 (40/250 vs 6/80). Chi-square: **p ≈ 0.085** (not significant at α=0.05). We treat this as a *pilot* comparison under repo-matched human sampling—not a causal claim—and note that Cursor/Devin/Jules cells remain small.

Interpretation. Expanding the taxonomy with R12–R15 (dangerous scopes, remote shell pipes, interactive triggers, `GITHUB_ENV` injection) and tightening R07/R10 signals raises gold coverage while keeping the validator **high-precision / high-recall / low-FPR** (P≈0.90, R≈0.96, FPR≈0.02). The after-only zizmor baseline again shows high recall with very high FPR (≈0.91). The before/after-diff zizmor baseline cuts FPR to ≈0.18 with moderate recall (≈0.74), remaining noisier than the policy validator. Numbers regenerate from `annotations/labels_aidev.csv` via `python -m evaluate.run_eval --out paper/manuscript/tables/field`.

7. Discussion

7.1 Diff-aware detection complements after-only scanning

The milestone results illustrate a structural point: security regression is a property of a *change*, not only of a final file. Removal of a CodeQL job can make the after file shorter and syntactically simpler while making the pipeline less safe. Unpinning an action can look like a routine version bump. Conversely, after-only zizmor scanning on our suite achieved high recall for the regression label only by flagging *all* pairs, including safe hardenings (FPR = 1.00), because residual snapshot findings are ubiquitous. In deployment, organizations should run both: static scanners for residual risk in the landing revision, and regression policies for PR gating focused on worsenings.

7.2 Practical implications for agent merge gates

As coding agents submit more PRs that touch `.github/workflows/`, merge queues should treat workflow paths as security-sensitive. A lightweight validator with tens of milliseconds of latency per file is compatible with CI. Policies can be tuned per organization—for example, forbidding `write-all`, requiring SHA pins for third-party actions, or blocking deletion of named security jobs without explicit waiver comments.

7.3 Relationship to 2026 agent-in-CI incidents

Microsoft and CSA reports [5], [6] show that agents running inside Actions can be coerced into leaking secrets. Our authoring-time regressions are complementary: an agent need not run in CI to harm security if it merges a weaker workflow. Defenses should therefore cover both runtime agent sandboxing and diff-time configuration policy.

7.4 What this milestone does and does not prove

This manuscript proves that (i) a concrete OWASP-mapped regression taxonomy (R01–R15) can be implemented with versioned formal predicates (policy v3), (ii) negative controls and diverse positives are detectable by the validator with FPR = 0.00 on the controlled suite, and (iii) after-only zizmor scanning, while recalling all gold regressions on that suite, yields FPR = 1.00 when naively used as a regression predictor—showing that snapshot findings are not

interchangeable with before/after worsenings. The AIDev field pilot (Section 6.6) further shows an overall 13.9% v3-audited regression rate (N = 330), a strong validator profile (P≈0.90, R≈0.96, FPR≈0.02), and a zizmor before/after-diff baseline with substantially lower FPR (≈0.18) than after-only (≈0.91). The pooled agent-versus-human rate gap (16.0% vs 7.5%) is directionally elevated but not statistically significant under v3 gold (χ^2 p≈0.085). Larger multi-agent cells and PR-level human matching remain desirable before strong epidemiological claims.

8. Threats to Validity

Construct validity. Our regression definition may diverge from organization-specific policies (for example, some teams intentionally use `pull_request_target` with careful hardening). Declarative rules make such disagreements explicit and tunable.

Internal validity. Benchmark fixtures were co-designed with rules, creating overfitting risk. We mitigate by including multiple negatives and non-worsening edits, but field audits are mandatory before strong claims.

External validity. N = 15 synthetic pairs cannot represent GitHub diversity across languages, org sizes, reusable workflows, and matrix builds. Agent labels on fixtures are not AIDev behavioral samples.

Baseline construct validity. Treating any zizmor after-file finding as a regression positive is a strict operationalization of after-only gating. We additionally evaluate set-difference of zizmor findings before versus after; on the field pilot this reduces FPR from ≈0.91 to ≈0.21 but remains above the policy validator's FPR.

Conclusion validity. Fisher p-values on a designed corpus are fragile. We report them transparently as harness outputs, not as confirmatory epidemiology.

Label validity. Milestone gold labels were curated with the fixtures. Future double annotation and agreement statistics (for example, Cohen's κ) are planned for field samples.

9. Conclusion and Future Work

We presented a milestone empirical study of security regressions in GitHub Actions workflow diffs associated with AI coding-agent settings. We introduced an OWASP-mapped taxonomy, implemented a policy-based before/after validator, and evaluated it on a controlled

fifteen-pair benchmark. The validator achieved F1 = 1.00 with FPR = 0.00, compared with F1 = 0.80 but FPR = 1.00 for after-only zizmor v1.31.1 used as a regression predictor, and ablation results showed that no single rule family alone recovers full recall. We also specified a reproducible protocol for scaling the study to AIDev.

Future work will proceed in four concrete steps: (1) enlarge sparse agent cells (Cursor/Jules/Devin) and strengthen PR-level human matching; (2) double-annotate stratified field labels and report inter-annotator agreement; (3) package the v2 validator as a GitHub Action suitable for merge protection on workflow paths; and (4) explore organization-tunable policy packs beyond the default trusted-owner list. With those steps, the manuscript can transition from milestone-plus-pilot evidence to journal-ready field epidemiology.

Figures

The following figures are generated from the reproducible evaluation pipeline (paper/manuscript/figures/).

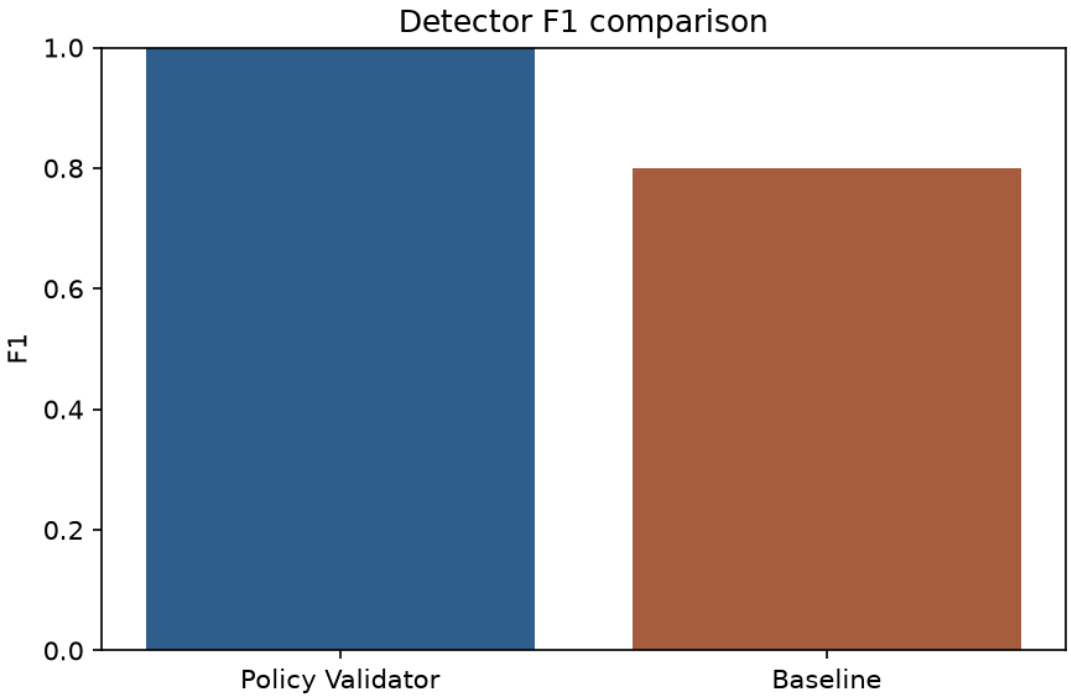


Figure 1. Detector F1 comparison (Policy Validator vs zizmor after-only baseline).

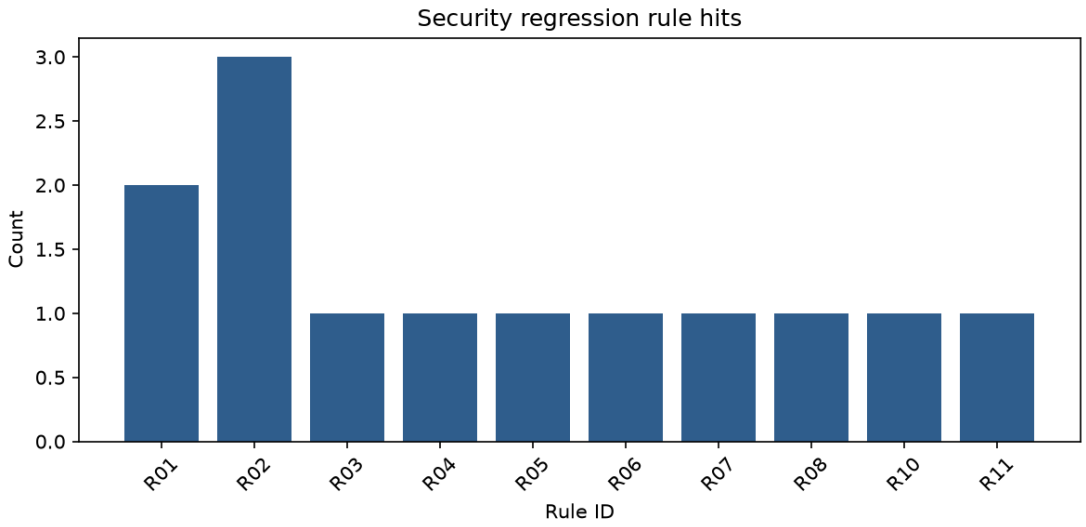


Figure 2. Security regression rule hit counts on the controlled benchmark.

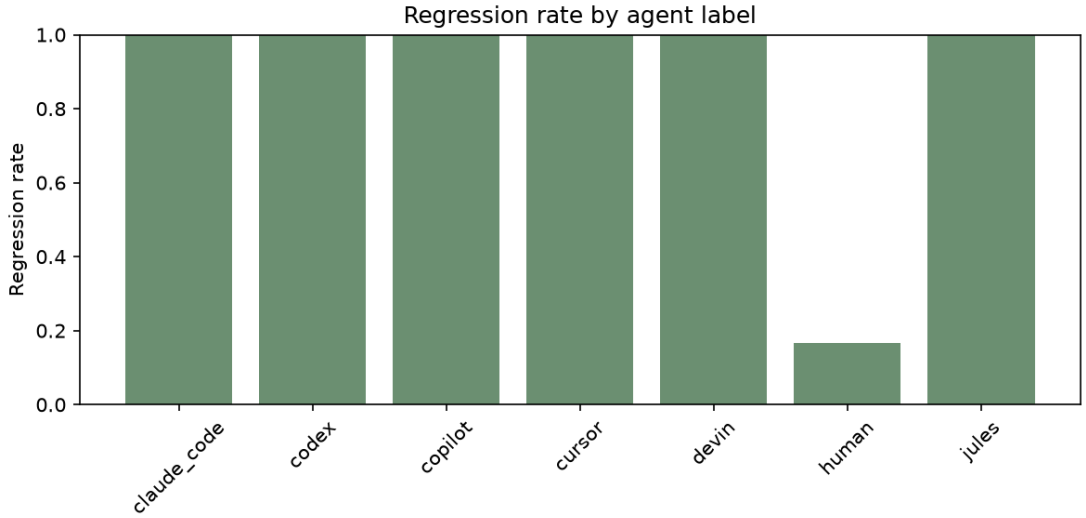


Figure 3. Regression rate by agent label on the controlled benchmark (illustrative).

Acknowledgments

The authors thank the maintainers of the AIDev dataset and the open-source CI security tooling community. Feedback from the academic supervisor will be acknowledged in the camera-ready version.

Funding: [To be filled — none if applicable].

Conflict of interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

AI writing disclosure (JAIAI): In order to correct and improve the academic writing of our paper, we have used the language model Cursor Agent (Composer).

Data and code availability: The companion repository contains `src/` (validator, extraction, evaluation), `policies/`, `data/sample/`, `annotations/`, scripts, and generated tables/figures under `paper/manuscript/`. Field AIDev-derived raw downloads are not redistributed; indexes and redacted pairs will be shared according to upstream licenses.

References

[1] H. Li, H. Zhang, and A. E. Hassan, "AIDev: Studying AI Coding Agents on GitHub," arXiv:2507.15003, 2025. Dataset: https://huggingface.co/datasets/hao-li/AIDev

[2] T. Ghaleb et al., "When AI Agents Touch CI/CD Configurations: Frequency and Success," MSR 2026 Mining Challenge, arXiv:2601.17413, 2026.

[3] "Reliability of AI Bots Footprints in GitHub Actions CI/CD Workflows," in *Proc. Mining Software Repositories Conf. (MSR)*, 2026, doi: 10.1145/3793302.3793569.

[4] zizmor project, "zizmor documentation," https://docs.zizmor.sh/ (accessed Oct. 21, 2026).

[5] Microsoft Threat Intelligence, "Securing CI/CD in an agentic world: Claude Code GitHub Action case," Microsoft Security Blog, Jun. 5, 2026.

[6] Cloud Security Alliance, "Prompt Injection in AI-Powered GitHub Actions," May 3, 2026.

[7] OWASP Foundation, "Top 10 CI/CD Security Risks," https://owasp.org/www-project-top-10-ci-cd-security-risks/

[8] GitHub, "Security hardening for GitHub Actions," GitHub Docs, https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions

[9] OWASP, "GitHub Actions Security Cheat Sheet," https://cheatsheetseries.owasp.org/cheatsheets/GitHub_Actions_Security_Cheat_Sheet.html

[10] GitHub, “Securely using pull_request_target,” GitHub Docs, https://docs.github.com/en/actions/reference/security/securely-using-pull_request_target

[11] M. Tamtaji, A. Ekrami Kivaj, and S. G. H. Tabatabaei, “The Role of Artificial Intelligence in Predicting Cyber Attack Patterns and Offering Solutions to Mitigate Attacks in ISMS Compliant Environments,” *Journal of Artificial Intelligence, Applications and Innovations*, vol. 1, no. 4, pp. 14–29, 2024.

[12] “Reviewing the Landscape of Security Anomaly Detection through Deep Learning Techniques,” *Journal of Artificial Intelligence, Applications and Innovations*, 2024.

[13] GitHub Security Lab, “Keeping your GitHub Actions and workflows secure” (series), https://securitylab.github.com/

[14] OpenSSF, “Scorecard,” https://github.com/ossf/scorecard

[15] OWASP, “Agent Security Regression Harness,” https://github.com/OWASP/Agent-Security-Regression-Harness (behavioral agent regressions; complementary scope).

[16] H. Li et al., AIDev dataset versions on Hugging Face (AIDev / AIDev-7.6M), 2025–2026.

[17] Taher-Ghaleb, “AIAgentsCI_CD-MSR2026 replication package,” https://github.com/Taher-Ghaleb/AIAgentsCI_CD-MSR2026

[18] rhysd, “actionlint,” https://github.com/rhysd/actionlint

[19] CISA / industry reporting on compromised GitHub Actions (e.g., tj-actions/changed-files, CVE-2025-30066), summarized in [6].

[20] GitHub, “Using the GITHUB_TOKEN in a workflow,” GitHub Docs.

[21] National Association of Artificial Intelligence of Iran / JAIAI, Author Guidelines, https://www.journalaiai.com/index.php/aiai/about/submissions

[22] Zenodo record for AIDev dataset materials, https://zenodo.org/records/16919051

Appendix A. Example policy rules and YAML fragments

A.1 Rule R01/R02 example (permissions)

Before:

```
permissions:
  contents: read
```

After:

```
permissions: write-all
```

Expected findings: R01 (escalation to write-all), R02 (explicit restriction removed).

A.2 Rule R03/R04 example (privileged PR checkout)

Before trigger: `pull_request`

After trigger: `pull_request_target` with:

```
- uses: actions/checkout@v4
  with:
    ref: ${{ github.event.pull_request.head.sha }}
```

Expected findings: R03, R04.

A.3 Rule R08 example (security gate removal)

Before includes a `codeql` job using `github/codeql-action/*`. After deletes that job and retains only tests. Expected finding: R08.

Full machine-readable rules are maintained in `policies/owasp_cicd_rules.yaml`.

Appendix B. System architecture

```
[Synthetic fixtures / AIDev PRs]
        | filter .github/workflows
        v
[Before YAML] ----\
```

```
                  >--> Signal extraction --> Diff --> Policy rules --> Findings
[After YAML] -----/
        |
        +--> Labels (gold)
        |
        +--> Metrics vs zizmor_after_only --> Tables/Figures --> Manuscript
```

Figures referenced in the main text are stored at:

Figure 1: `paper/manuscript/figures/method_f1.png`

Figure 2: `paper/manuscript/figures/rule_counts.png`

Figure 3: `paper/manuscript/figures/agent_rates.png`

End of milestone manuscript v0.2-full.

Generated from paper/manuscript/main.md for advisor review / JAIAI preparation. Replace [To be filled] fields and paste into the official JAIAI English DOC template before submission.